University of Lahore

LAHORE, PAKISTAN

INTRODUCTION TO MACHINE LEARNING

Project Proposal

Telco Customer Churn Prediction



Predicting customer churn for better retention and stronger relationships.

COURSE	:	Introduction to Machine Learning
PROJECT TITLE	:	Telco Customer Churn Prediction
SUBMITTED BY	:	**ALI HAIDER** **70137343**
GROUP MEMBERS	:	1. M Umair Manzoor (70151873) 2. Ahmad Raza (70146602) 3. Samia Shahzadii(70148797)
SUBMISSION DATE	:	—

1. Introduction

Customer churn prediction is an important application of machine learning. It helps companies identify customers who may discontinue their services. By identifying these customers, companies can take steps to improve customer satisfaction and retention.

The purpose of this project is to predict customer churn using the Telco Customer Churn dataset. Three machine learning classification algorithms, Naive Bayes, Decision Tree, and Logistic Regression, are evaluated and compared.

The performance of these models is assessed using accuracy, F1-score, and five-fold cross-validation F1-score.

2. Objectives

The main objectives of this project are:

1. To understand and prepare the customer churn dataset for machine learning.
2. To train three classification models.
3. To evaluate model performance using accuracy and F1-score.
4. To compare the models using five-fold cross-validation.
5. To identify the model that performs best among the three tested algorithms.

3. Dataset Description

This project uses the Telco Customer Churn dataset, which contains customer information related to telecommunications services.

The target variable is `Churn`, which has two possible values: `Yes` and `No`.

- **Yes:** The customer has discontinued the service.
- **No:** The customer has continued the service.

The dataset contains customer-related features such as tenure, contract type, internet services, monthly charges, and total charges.

These features help the machine learning models learn patterns associated with customer churn.

4. Data Preprocessing

Data preprocessing is an important step because machine learning algorithms require data in a suitable format.

The preprocessing steps considered in this project include:

4.1 Handling Missing or Invalid Values

The `TotalCharges` column may contain blank or non-numeric values. These values must be handled appropriately before model training.

4.2 Encoding Categorical Variables

Categorical features must be converted into numerical representations so that the machine learning algorithms can process them.

4.3 Feature Scaling

Feature scaling may be applied where appropriate to ensure that numerical features are represented on suitable scales.

4.4 Splitting the Dataset

The dataset is divided into training and testing sets. The training set is used to train the models, while the testing set is used to evaluate their performance on unseen data.

Note: Include only the preprocessing steps that you actually performed in your Colab notebook.

5. Machine Learning Algorithms

Three classification algorithms were used in this project.

5.1 Naive Bayes

Naive Bayes is a probabilistic classification algorithm based on Bayes' theorem. It assumes conditional independence between features given the target class.

It is computationally efficient and can be useful for classification tasks. However, its performance depends on how well its assumptions fit the dataset.

5.2 Decision Tree

A Decision Tree classifies data by splitting it according to feature values. Each internal node represents a decision, each branch represents an outcome, and each leaf represents a predicted class.

Decision Trees are relatively easy to interpret and can capture nonlinear relationships between features.

5.3 Logistic Regression

Logistic Regression is a classification algorithm that estimates the probability of an observation belonging to a particular class.

Although its name includes the word regression, it is commonly used for binary classification problems such as customer churn prediction.

In this project, Logistic Regression achieved the highest test accuracy and cross-validation F1-score among the three tested models.

6. Model Evaluation Metrics

The models were evaluated using the following metrics.

6.1 Accuracy

Accuracy measures the proportion of all predictions that are correct.

$$Accuracy = \frac{TP+TN}{TP+TN+FP+FN}$$

Here, TP represents true positives, TN represents true negatives, FP represents false positives, and FN represents false negatives.

6.2 Precision

Precision measures how many customers predicted as churners actually belong to the positive class.

$$Precision = \frac{TP}{TP+FP}$$

6.3 Recall

Recall measures how many actual positive cases are correctly identified by the model.

$$Recall = \frac{TP}{TP+FN}$$

6.4 F1-Score

The F1-score is the harmonic mean of precision and recall.

$$F1 = 2 \times \frac{Precision \times Recall}{Precision+Recall}$$

It is useful when both false positives and false negatives matter.

In this project, `Yes` is treated as the positive class when calculating the reported F1-scores.

6.5 Five-Fold Cross-Validation

Five-fold cross-validation divides the dataset into five parts. The model is trained and evaluated across five iterations, using a different fold for validation in each iteration.

The average F1-score across the five folds provides an additional measure of model performance.

7. Experimental Results

The models were evaluated using the testing data. The following results were obtained from the Colab outputs.

7.1 Test Set Performance

Model	Accuracy	F1-Score
Naive Bayes	28.03%	40.42%
Decision Tree	77.57%	53.25%
Logistic Regression	82.40%	63.95%

Note: The F1-scores are for the positive class, `Yes`, as specified in the code. Accuracy and F1-score measure different aspects of performance.

7.2 Cross-Validation Results

The following average F1-scores were obtained using five-fold cross-validation.

Model	Five-Fold CV F1-Score

Naive Bayes	40.90%
Decision Tree	54.09%
Logistic Regression	59.67%

These values are calculated from the outputs provided by the Colab notebook.

7.3 Interpretation of Results

Naive Bayes: The model achieved 28.03% accuracy and a 40.42% test F1-score. Its five-fold cross-validation F1-score was 40.90%. These results indicate comparatively weaker performance among the three models.

Decision Tree: The model achieved 77.57% accuracy and a 53.25% test F1-score. Its cross-validation F1-score was 54.09%, showing stronger performance than Naive Bayes.

Logistic Regression: The model achieved 82.40% accuracy and a 63.95% test F1-score. Its cross-validation F1-score was 59.67%, making it the strongest of the three models according to the reported results.

The difference between test F1-score and cross-validation F1-score indicates that model performance can vary across data splits. Further evaluation may help explain this variation.

8. Gini Impurity Calculation

Gini impurity measures how mixed the classes are within a group. It is commonly used by Decision Trees to evaluate potential splits.

The formula is:

$$Gini = 1 - \sum_{i=1}^{k} p_i^2$$

Here, p_i represents the proportion of observations belonging to class i.

8.1 Parent Node Gini Impurity

Consider an illustrative example containing 10 customers:

- Churn = Yes: 4 customers
- Churn = No: 6 customers

The Gini impurity is:

$$Gini(parent) = 1 - \left(\frac{4}{10}\right)^2 - \left(\frac{6}{10}\right)^2$$

$$= 1 - 0.16 - 0.36$$

\[\boxed{Gini(parent)=0.48} \]

8.2 Gini Impurity After Splitting

Suppose the customers are divided into two groups based on contract type.

Group	Churn = Yes	Churn = No	Total
Month-to-month	4	2	6
Long-term contract	0	4	4

Total	4	6	10

For the month-to-month group:

$$Gini = 1 - \left(\frac{4}{6}\right)^2 - \left(\frac{2}{6}\right)^2$$

\[\boxed{Gini=0.4444} \]

For the long-term contract group:

$$Gini = 1 - \left(\frac{0}{4}\right)^2 - \left(\frac{4}{4}\right)^2$$

\[\boxed{Gini=0} \]

8.3 Weighted Gini Impurity

The weighted Gini impurity after splitting is:

$$Gini(split) = \frac{6}{10}(0.4444) + \frac{4}{10}(0)$$

\[\boxed{Gini(split)=0.2667} \]

8.4 Gini Reduction

The reduction in impurity is:

$$Gini\ Reduction = 0.48 - 0.2667$$

\[\boxed{Gini\ Reduction=0.2133} \]

This illustrative example shows that a split can reduce Gini impurity by creating groups with more concentrated class distributions.

Important: The counts in this section are hypothetical and are used only to demonstrate the calculation. They are not verified counts from the actual dataset or a split performed by the trained Decision Tree.

9. Discussion

The experimental results show that Logistic Regression performed best among the three tested algorithms.

It achieved the highest test accuracy of 82.40% and the highest test F1-score of 63.95%. Its five-fold cross-validation F1-score was also the highest, at 59.67%.

Decision Tree performed second best, while Naive Bayes produced the weakest results in the comparison.

These findings suggest that Logistic Regression is a promising model for this particular experiment. However, accuracy alone is not sufficient to determine whether a churn prediction model is suitable for real-world use.

Customer churn datasets may contain an imbalance between the `Yes` and `No` classes. Therefore, precision, recall, the confusion matrix, and ROC-AUC should also be examined before drawing stronger conclusions.

10. Conclusion

This project compared Naive Bayes, Decision Tree, and Logistic Regression for customer churn prediction using the Telco Customer Churn dataset.

The results showed that Logistic Regression achieved 82.40% test accuracy, a 63.95% test F1-score, and a 59.67% five-fold cross-validation F1-score.

Decision Tree achieved 77.57% test accuracy and a 54.09% cross-validation F1-score. Naive Bayes achieved 28.03% test accuracy and a 40.90% cross-validation F1-score.

Based on these results, Logistic Regression was the best-performing model among the three evaluated algorithms.

The project demonstrates how machine learning can be applied to customer churn prediction and how different evaluation metrics can be used to compare classification models. Additional evaluation and model tuning could further improve the reliability of the predictions.

11. References

1. Scikit-learn Developers. *Scikit-learn: Machine Learning in Python.* https://scikit-learn.org/
2. Scikit-learn Developers. *Model Evaluation: Quantifying the Quality of Predictions.* https://scikit-learn.org/stable/modules/model_evaluation.html
3. Scikit-learn Developers. *Cross-validation: Evaluating Estimator Performance.* https://scikit-learn.org/stable/modules/cross_validation.html
4. IBM. *Telco Customer Churn Dataset.* https://www.kaggle.com/datasets/blastchar/telco-customer-churn